
Day 1 – Foundations of Java

- **History & Features of Java**

- Java was created by Sun Microsystems in 1995.
- Key features: **Object-Oriented Programming**, **platform independence** (write once, run anywhere), **JVM** (Java Virtual Machine), **JDK** (Java Development Kit), **JRE** (Java Runtime Environment).

- **Data Types**

- **Primitive types**: int, float, char, boolean, etc.
- **Non-primitive types**: Strings, arrays, classes.
- **Variable declaration** and initialization.
- **Type casting**: implicit (widening) and explicit (narrowing).

- **Object-Oriented Principles**

- **Encapsulation**: hiding data with access modifiers.
- **Inheritance**: reusing code from parent classes.
- **Polymorphism**: same method, different behavior.
- **Abstraction**: hiding implementation details.

- **Classes and Objects**

- **Class structure**: fields, methods, constructors.
- **Object creation** using new keyword.
- **Instance variables** vs local variables.

- **Constructors**

- **Default constructor** (no arguments).
- **Parameterized constructor** (with arguments).
- **Constructor overloading**.

Day 2 – Deep Dive into OOP

- **Inheritance**: **Single inheritance**, **multilevel inheritance**, using **super keyword**.

- Polymorphism: [Method overloading](#) vs [method overriding](#).
 - Abstraction: [Abstract classes](#) and [interfaces](#).
 - Encapsulation: [Access modifiers](#) (public, private, protected), [encapsulating data](#) with getters/setters.
-

Day 3 – Core Java Concepts

- this and super: [Referring current object](#), [accessing parent class methods](#).
 - Static Members: [Static variables](#), [static methods](#), [static blocks](#).
 - Final Keyword: [Final variables](#), [final methods](#), [final classes](#).
 - Arrays: [One-dimensional arrays](#), [multidimensional arrays](#), [array traversal](#).
 - Strings: [String class](#), [immutability](#), methods like [length](#), [substring](#), [equals](#).
-

Day 4 – Strings & Exceptions

- StringBuffer & StringBuilder: [Mutability](#), methods like [append](#), [insert](#), [delete](#).
 - Types of Exceptions: [Checked exceptions](#) vs [unchecked exceptions](#).
 - try-catch Block: [Single try-catch](#), [multiple catch](#), [finally block](#).
 - throw and throws: [Manually throwing exceptions](#), [propagating exceptions](#).
-

Day 5 – Collections (Part 1)

- Custom Exceptions: [Creating custom exceptions](#).
 - Introduction to Collections: [Collection hierarchy](#), [interfaces](#) (List, Set, Map).
 - List Interface: [ArrayList](#), [LinkedList](#), [Vector](#), [Stack](#).
-

Day 6 – Collections (Part 2)

- Set Interface: [HashSet](#), [LinkedHashSet](#), [TreeSet](#).
 - Map Interface: [HashMap](#), [LinkedHashMap](#), [TreeMap](#), [Hashtable](#).
 - Iterator: [Iterator](#), [ListIterator](#).
-

Day 7 – Threads & Streams

- **Comparable & Comparator:** Sorting with Comparable, sorting with Comparator.
 - **Threads:** Thread vs Process, thread lifecycle.
 - **Creating Threads:** Extending Thread class, implementing Runnable interface.
 - **Thread Methods:** sleep(), join(), start(), run().
 - **Character Streams:** FileReader, FileWriter, BufferedReader, BufferedWriter.
-

Day 8 – File Handling & JDBC

- **File Handling:** File class, FileInputStream, FileOutputStream.
 - **JDBC API Overview:** JDBC architecture, Driver Manager, Statement, PreparedStatement.
 - **CRUD Operations:** Insert, Update, Delete, Select.
 - **Transactions:** Commit, Rollback, Savepoints.
-

Day 9 – Modern Java Features

- **Lambda Expressions:** Lambda syntax, functional interfaces.
 - **Stream API:** filter, map, reduce.
 - **Optional Class:** Avoiding NullPointerException.
 - **Generics:** Generic classes, methods, wildcards.
-

Day 10 – Advanced Concepts

- **Annotations:** @Override, @Deprecated, custom annotations.
 - **Design Patterns:** Singleton, Factory, Observer, Strategy.
-

Day 11 & 12 – Consolidation

- **Java 8 Features:** Streams, Optionals, asynchronous programming, parallel programming.

- **Collections Framework:** Deep dive into **data structures**.
 - **Generics:** Advanced use cases.
-

Day 1 – Foundations of Java

Topic 1: History & Features of Java

Why: Java was created to solve the problem of software running on different platforms.

What: It's an object-oriented, platform-independent language.

How: Programs are compiled into bytecode, which runs on the JVM.

When: Use Java when you need cross-platform applications (desktop, web, mobile).

Who: Developers across industries — banking, Android apps, enterprise systems.

Code Example:

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello, Java!");  
  
    }  
  
}
```

Exercise:

Write a program that prints your name and age.

Quiz:

Q1: What makes Java platform-independent?

- a) JDK
 - b) JVM
 - c) Compiler
 - d) OS
-

Topic 2: Data Types

Why: To store and manipulate data efficiently.

What: Primitive (int, float, char, boolean) and non-primitive (String, arrays).

How: Declare variables with type.

When: Use primitive for simple values, non-primitive for complex structures.

Who: Every Java programmer uses them daily.

Code Example:

```
public class DataTypesDemo {  
    public static void main(String[] args) {  
        int age = 25;  
        double salary = 50000.50;  
        char grade = 'A';  
        boolean isActive = true;  
        String name = "John";  
  
        System.out.println(name + " is " + age + " years old.");  
    }  
}
```

Exercise:

Declare variables for a student's name, roll number, and marks. Print them.

Quiz:

Q2: Which of these is a non-primitive type?

- a) int
- b) char
- c) String
- d) boolean

Topic 3: Object-Oriented Principles

Why: To organize code into reusable, modular components.

What: Encapsulation, Inheritance, Polymorphism, Abstraction.

How: Use classes, objects, access modifiers, inheritance keywords.

When: Large projects with complex relationships.

Who: Teams building scalable applications.

Code Example (Encapsulation):

```
class Student {  
    private String name;  
  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}  
  
public class TestEncapsulation {  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.setName("Alice");  
        System.out.println(s.getName());  
    }  
}
```

Exercise:

Create a BankAccount class with private balance and methods to deposit/withdraw.

Quiz:

Q3: Which principle hides implementation details?

- a) Inheritance
- b) Polymorphism
- c) Abstraction
- d) Encapsulation

Topic 4: Classes and Objects

Why: Classes define structure, objects bring them to life.

What: Class = blueprint, Object = instance.

How: Use class keyword, create objects with new.

When: Anytime you model real-world entities.

Who: Every Java developer.

Code Example:

```
class Car {  
    String color;  
    int speed;  
}
```

```
public class TestCar {  
    public static void main(String[] args) {  
        Car myCar = new Car();  
        myCar.color = "Red";  
        myCar.speed = 120;  
        System.out.println("Car color: " + myCar.color);  
    }  
}
```

Exercise:

Create a Book class with title and author. Create two objects and print details.

Quiz:

Q4: What keyword is used to create an object?

- a) class
- b) new
- c) object
- d) create

Topic 5: Constructors

Why: To initialize objects.

What: Special methods with same name as class.

How: Default, parameterized, overloaded constructors.

When: When you want to set initial values.

Who: Developers working with object initialization.

Code Example:

```
class Employee {  
  
    String name;  
  
    int id;  
  
  
    // Default constructor  
    Employee() {  
        name = "Unknown";  
        id = 0;  
    }  
  
    // Parameterized constructor  
    Employee(String n, int i) {  
        name = n;  
        id = i;  
    }  
}  
  
public class TestEmployee {  
    public static void main(String[] args) {  
        Employee e1 = new Employee();  
    }  
}
```



```
Employee e2 = new Employee("John", 101);

System.out.println(e1.name + " " + e1.id);

System.out.println(e2.name + " " + e2.id);

}

}
```

Exercise:

Create a Laptop class with constructors to initialize brand and price.

Quiz:

Q5: What is true about constructors?

- a) They return values
- b) They have same name as class
- c) They can be static
- d) They must be private

 Day 2 – Deep Dive into OOP

Topic 1: Inheritance

Why: To reuse code and establish relationships between classes.

What: Mechanism where one class (child) derives properties/methods from another (parent).

How: Use extends keyword.

When: When classes share common behavior.

Who: Developers building hierarchical models (e.g., Employee → Manager).

Code Example (Single Inheritance):

```
class Animal {

    void eat() {

        System.out.println("Eating...");

    }

}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking...");  
    }  
}
```

```
public class TestInheritance {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat(); // inherited  
        d.bark(); // own method  
    }  
}
```

Code Example (Multilevel Inheritance with super):

```
class Vehicle {  
    Vehicle() {  
        System.out.println("Vehicle created");  
    }  
}
```

```
class Car extends Vehicle {  
    Car() {  
        super(); // calls parent constructor  
        System.out.println("Car created");  
    }  
}
```

```
class SportsCar extends Car {  
    SportsCar() {  
        super(); // calls Car constructor  
        System.out.println("SportsCar created");  
    }  
}
```

```
public class TestMultilevel {  
    public static void main(String[] args) {  
        SportsCar sc = new SportsCar();  
    }  
}
```

Exercise:

Create a Person class with name. Extend it to Employee with salary. Extend further to Manager with department. Print all details.

Quiz:

Q1: Which keyword is used to call parent class constructor?

- a) this
- b) super
- c) parent
- d) extends

Topic 2: Polymorphism

Why: To allow one interface to represent different behaviors.

What: Method overloading (compile-time) and method overriding (runtime).

How: Overloading = same method name, different parameters. Overriding = same method signature in child class.

When: Overloading for flexibility, overriding for dynamic behavior.

Who: Developers designing APIs or frameworks.

Code Example (Method Overloading):

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
    double add(double a, double b) {  
        return a + b;  
    }  
}  
  
public class TestOverloading {  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
        System.out.println(c.add(5, 10));  
        System.out.println(c.add(5.5, 10.5));  
    }  
}
```

Code Example (Method Overriding):

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
}  
}
```

```
public class TestOverriding {  
    public static void main(String[] args) {  
        Animal a = new Dog(); // runtime polymorphism  
        a.sound();  
    }  
}
```

Exercise:

Create a Shape class with a draw() method. Override it in Circle and Rectangle.

Quiz:

Q2: Which type of polymorphism uses method overriding?

- a) Compile-time
- b) Runtime
- c) Static
- d) None

Topic 3: Abstraction

Why: To hide implementation details and show only essential features.

What: Achieved using abstract classes and interfaces.

How: Use abstract keyword or interface.

When: When designing frameworks or contracts.

Who: Developers building reusable libraries.

Code Example (Abstract Class):

```
abstract class Animal {  
    abstract void sound();  
}
```

```
class Cat extends Animal {  
    void sound() {  
        System.out.println("Meow");  
    }  
}
```

```
public class TestAbstract {  
    public static void main(String[] args) {  
        Animal a = new Cat();  
        a.sound();  
    }  
}
```

Code Example (Interface):

```
interface Drawable {  
    void draw();  
}
```

```
class Circle implements Drawable {  
    public void draw() {  
        System.out.println("Drawing Circle");  
    }  
}
```

```
public class TestInterface {  
    public static void main(String[] args) {  
        Drawable d = new Circle();  
        d.draw();  
    }  
}
```

```
}
```

Exercise:

Create an interface Playable with method play(). Implement it in Guitar and Piano.

Quiz:

Q3: Which keyword is used to define an abstract class?

- a) interface
- b) abstract
- c) class
- d) implements

Topic 4: Encapsulation

Why: To protect data and control access.

What: Wrapping data and methods together.

How: Use private variables + public getters/setters.

When: When sensitive data must be controlled.

Who: Developers working with secure applications.

Code Example:

```
class Account {  
  
    private double balance;  
  
    public void deposit(double amount) {  
        balance += amount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

```
public class TestEncapsulation {  
    public static void main(String[] args) {  
        Account acc = new Account();  
        acc.deposit(1000);  
        System.out.println("Balance: " + acc.getBalance());  
    }  
}
```

Exercise:

Create a Student class with private marks. Provide getter/setter methods.

Quiz:

Q4: Which access modifier ensures data hiding?

- a) public
- b) private
- c) protected
- d) default

 Day 3 – Core Java Concepts

Topic 1: this and super

Why: To differentiate between current object and parent class references.

What:

- this → refers to the current object.
- super → refers to parent class members.

How: Use inside constructors/methods.

When: When variables/methods are shadowed or parent methods need to be accessed.

Who: Developers working with inheritance and object initialization.

Code Example (this keyword):

```
class Student {
```



```
String name;

Student(String name) {
    this.name = name; // refers to current object
}

void display() {
    System.out.println("Name: " + this.name);
}
}
```

```
public class TestThis {
    public static void main(String[] args) {
        Student s = new Student("Alice");
        s.display();
    }
}
```

Code Example (super keyword):

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}
```

```
class Dog extends Animal {
    void sound() {
        super.sound(); // calls parent method
        System.out.println("Dog barks");
    }
}
```

```
public class TestSuper {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

Exercise:

Create a Vehicle class with a start() method. Extend it in Car and override start() but also call parent method using super.

Quiz:

Q1: Which keyword refers to the current object?

- a) super
- b) this
- c) parent
- d) self

Topic 2: Static Members

Why: To share variables/methods across all objects.

What: Static variables, methods, and blocks.

How: Use static keyword.

When: When data/methods should be common for all instances.

Who: Developers creating utility classes or constants.

Code Example:

```
class Counter {  
    static int count = 0;  
    Counter() {  
        count++;  
        System.out.println("Object count: " + count);  
    }  
}
```

```
}  
}
```

```
public class TestStatic {  
  
    public static void main(String[] args) {  
  
        new Counter();  
  
        new Counter();  
  
        new Counter();  
  
    }  
}
```

Exercise:

Create a MathUtil class with a static method square(int x) that returns the square of a number.

Quiz:

Q2: Which block executes before main method?

- a) static block
- b) constructor
- c) instance block
- d) method

Topic 3: Final Keyword

Why: To prevent modification.

What: Final variables (constants), final methods (cannot be overridden), final classes (cannot be inherited).

How: Use final keyword.

When: When immutability or restriction is needed.

Who: Developers ensuring security and stability.

Code Example:

```
final class Constants {
```

```
    final double PI = 3.14159;
}

class TestFinal {

    public static void main(String[] args) {

        Constants c = new Constants();

        System.out.println("PI: " + c.PI);

    }

}
```

Exercise:

Create a Bank class with a final method `getInterestRate()`. Try overriding it in a subclass.

Quiz:

Q3: What happens if you try to extend a final class?

- a) Compilation error
- b) Runtime error
- c) Works fine
- d) Warning only

Topic 4: Arrays

Why: To store multiple values in a single variable.

What: One-dimensional and multidimensional arrays.

How: Declare with `[]`.

When: When handling collections of fixed size.

Who: Developers working with structured data.

Code Example (1D Array):

```
public class TestArray {

    public static void main(String[] args) {

        int[] numbers = {10, 20, 30};

        for(int n : numbers) {
```

```
        System.out.println(n);
    }
}
}
```

Code Example (2D Array):

```
public class Test2DArray {
    public static void main(String[] args) {
        int[][] matrix = {{1,2},{3,4}};
        for(int i=0; i<matrix.length; i++) {
            for(int j=0; j<matrix[i].length; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Exercise:

Create an array of 5 student names and print them using a loop.

Quiz:

Q4: What is the index of the first element in an array?

- a) 0
- b) 1
- c) -1
- d) Depends

Topic 5: Strings

Why: To represent and manipulate text.

What: String class, immutability, common methods.

How: Use String objects and methods.

When: Anytime text processing is needed.

Who: Every Java developer.

Code Example:

```
public class TestString {  
  
    public static void main(String[] args) {  
  
        String s = "Hello";  
  
        System.out.println("Length: " + s.length());  
  
        System.out.println("Substring: " + s.substring(1,4));  
  
        System.out.println("Equals: " + s.equals("Hello"));  
  
    }  
}
```

Exercise:

Write a program to check if a given string is a palindrome.

Quiz:

Q5: Are Strings mutable in Java?

- a) Yes
- b) No
- c) Sometimes
- d) Depends

 Day 4 – Strings & Exceptions

Topic 1: StringBuffer & StringBuilder

Why: Strings in Java are immutable. For frequent modifications, StringBuffer and StringBuilder are used.

What:

- StringBuffer → thread-safe, synchronized.

- `StringBuilder` → faster, not synchronized.

How: Use methods like `append()`, `insert()`, `delete()`.

When: Use `StringBuffer` in multithreaded environments, `StringBuilder` in single-threaded.

Who: Developers handling dynamic text manipulation.

Code Example:

```
public class TestStringBuffer {  
  
    public static void main(String[] args) {  
  
        StringBuffer sb = new StringBuffer("Hello");  
  
        sb.append(" World");  
  
        sb.insert(5, " Java");  
  
        sb.delete(0, 5);  
  
        System.out.println(sb); // Output: Java World  
  
    }  
}
```

Exercise:

Use `StringBuilder` to reverse a string entered by the user.

Quiz:

Q1: Which is faster but not thread-safe?

- a) `StringBuffer`
- b) `StringBuilder`
- c) `String`
- d) None

Topic 2: Types of Exceptions

Why: To handle runtime errors gracefully.

What:

- **Checked exceptions** → must be handled at compile time (e.g., `IOException`).

- **Unchecked exceptions** → occur at runtime (e.g., NullPointerException).
How: Use try-catch or declare with throws.
When: Whenever error-prone code is written.
Who: Every developer dealing with input/output, user data, or external systems.

Code Example:

```
import java.io.FileReader;

import java.io.IOException;

public class TestException {

    public static void main(String[] args) {

        try {

            FileReader fr = new FileReader("file.txt"); // Checked exception

        } catch (IOException e) {

            System.out.println("File not found!");

        }

        String s = null;

        System.out.println(s.length()); // Unchecked exception

    }

}
```

Exercise:

Write a program that tries to divide a number by zero and handle the exception.

Quiz:

Q2: Which type of exception must be handled at compile time?

- a) Checked
- b) Unchecked
- c) Runtime
- d) None

Topic 3: try-catch Block

Why: To prevent program crashes.

What: Handles exceptions using try, catch, and finally.

How: Place risky code in try, handle in catch, cleanup in finally.

When: Whenever exceptions may occur.

Who: Developers writing robust applications.

Code Example:

```
public class TestTryCatch {  
  
    public static void main(String[] args) {  
  
        try {  
  
            int a = 10 / 0;  
  
        } catch (ArithmeticException e) {  
  
            System.out.println("Cannot divide by zero!");  
  
        } finally {  
  
            System.out.println("Finally block executed");  
  
        }  
  
    }  
  
}
```

Exercise:

Write a program with multiple catch blocks to handle `ArrayIndexOutOfBoundsException` and `NullPointerException`.

Quiz:

Q3: Which block always executes?

- a) try
 - b) catch
 - c) finally
 - d) throw
-

Topic 4: throw and throws

Why: To manually throw exceptions or declare them.

What:

- throw → used to explicitly throw an exception.
- throws → declares exceptions a method may throw.

How: Use throw new Exception() or throws Exception.

When: When custom error handling is needed.

Who: Developers designing APIs or custom error flows.

Code Example:

```
class TestThrow {  
  
    static void validateAge(int age) throws Exception {  
        if(age < 18) {  
            throw new Exception("Not eligible to vote");  
        } else {  
            System.out.println("Eligible to vote");  
        }  
    }  
}  
  
    public static void main(String[] args) {  
        try {  
            validateAge(15);  
        } catch(Exception e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Exercise:

Create a method that throws an exception if a password length is less than 6 characters.

Quiz:

Q4: Which keyword is used to declare exceptions in method signature?

- a) throw
- b) throws
- c) try
- d) catch

Day 5 – Custom Exceptions & Collections (Part 1)

Topic 1: Custom Exceptions

Why: Built-in exceptions don't always cover specific business logic errors.

What: User-defined exceptions created by extending `Exception` or `RuntimeException`.

How: Create a class extending `Exception`, throw it with `throw new`, and handle with `try-catch`.

When: When you need meaningful error messages for domain-specific problems.

Who: Developers building robust applications with clear error handling.

Code Example:

```
class InvalidAgeException extends Exception {  
    InvalidAgeException(String message) {  
        super(message);  
    }  
}  
  
public class TestCustomException {  
    static void validateAge(int age) throws InvalidAgeException {  
        if(age < 18) {  
            throw new InvalidAgeException("Age must be 18 or above");  
        } else {  
            System.out.println("Valid age");  
        }  
    }  
}
```

```

    }

    public static void main(String[] args) {
        try {
            validateAge(15);
        } catch(InvalidAgeException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
    }
}

```

Exercise:

Create a custom exception `InsufficientBalanceException` for a banking application when withdrawal exceeds balance.

Quiz:

Q1: How do you create a custom exception?

- a) Extend Throwable
- b) Extend Exception or RuntimeException
- c) Use throw keyword only
- d) Extend Error

Topic 2: Introduction to Collections

Why: Arrays have fixed size; collections provide dynamic storage.

What: Java Collections Framework (JCF) provides interfaces like List, Set, Map.

How: Use classes like ArrayList, HashSet, HashMap.

When: When handling groups of objects with flexibility.

Who: Developers working with data structures.

Code Example:

```
import java.util.*;
```

```

public class TestCollections {

    public static void main(String[] args) {

        List<String> names = new ArrayList<>();

        names.add("Alice");

        names.add("Bob");

        names.add("Charlie");


        for(String name : names) {

            System.out.println(name);

        }

    }

}

```

Exercise:

Create a list of integers and print only even numbers.

Quiz:

Q2: Which interface represents a group of objects with duplicates allowed?

- a) Set
- b) List
- c) Map
- d) Collection

Topic 3: List Interface

Why: To store ordered elements with duplicates allowed.

What: Implementations include ArrayList, LinkedList, Vector, Stack.

How: Use add(), remove(), get(), size().

When: When order matters or duplicates are needed.

Who: Developers handling ordered datasets.

Code Example (ArrayList):

```

import java.util.*;

```

```

public class TestArrayList {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>();
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Mango");

        System.out.println(fruits.get(1)); // Banana
    }
}

```

Code Example (LinkedList):

```

import java.util.*;

public class TestLinkedList {
    public static void main(String[] args) {
        LinkedList<Integer> numbers = new LinkedList<>();
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);

        numbers.removeFirst();
        System.out.println(numbers);
    }
}

```

Code Example (Stack):

```

import java.util.*;

```

```
public class TestStack {  
    public static void main(String[] args) {  
        Stack<String> stack = new Stack<>();  
        stack.push("A");  
        stack.push("B");  
        stack.push("C");  
  
        System.out.println(stack.pop()); // C  
    }  
}
```

Exercise:

Create a Vector of student names and print them using an iterator.

Quiz:

Q3: Which List implementation allows fast random access?

- a) ArrayList
- b) LinkedList
- c) Stack
- d) Vector

 Day 6 – Collections (Part 2)

Topic 1: Set Interface

Why: To store unique elements without duplicates.

What: A collection that does not allow duplicate values. Implementations include HashSet, LinkedHashSet, and TreeSet.

How: Use add(), remove(), contains().

When: When uniqueness of elements is required.

Who: Developers working with distinct datasets (e.g., IDs, usernames).

Code Example (HashSet):

```
import java.util.*;

public class TestHashSet {

    public static void main(String[] args) {

        HashSet<String> set = new HashSet<>();

        set.add("Apple");

        set.add("Banana");

        set.add("Apple"); // duplicate ignored


        System.out.println(set); // Output: [Apple, Banana]

    }

}
```

Code Example (TreeSet):

```
import java.util.*;

public class TestTreeSet {

    public static void main(String[] args) {

        TreeSet<Integer> numbers = new TreeSet<>();

        numbers.add(30);

        numbers.add(10);

        numbers.add(20);


        System.out.println(numbers); // Output: [10, 20, 30] (sorted)

    }

}
```

Exercise:

Create a LinkedHashSet of city names and print them in insertion order.

Quiz:

Q1: Which Set implementation maintains sorted order?

- a) HashSet
- b) LinkedHashSet
- c) TreeSet
- d) Vector

Topic 2: Map Interface

Why: To store key-value pairs.

What: A collection that maps keys to values. Implementations include HashMap, LinkedHashMap, TreeMap, Hashtable.

How: Use put(), get(), remove().

When: When you need fast lookups by key.

Who: Developers working with dictionaries, configurations, or mappings.

Code Example (HashMap):

```
import java.util.*;

public class TestHashMap {

    public static void main(String[] args) {

        HashMap<Integer, String> map = new HashMap<>();

        map.put(1, "Alice");

        map.put(2, "Bob");

        System.out.println(map.get(1)); // Alice

    }

}
```

Code Example (LinkedHashMap):

```
import java.util.*;

public class TestLinkedHashMap {
```

```
public static void main(String[] args) {  
    LinkedHashMap<Integer, String> map = new LinkedHashMap<>();  
    map.put(1, "One");  
    map.put(2, "Two");  
    map.put(3, "Three");  
  
    System.out.println(map); // Maintains insertion order  
}  
}
```

Exercise:

Create a TreeMap of student roll numbers and names. Print them in ascending order of roll numbers.

Quiz:

Q2: Which Map implementation is synchronized?

- a) HashMap
- b) LinkedHashMap
- c) TreeMap
- d) Hashtable

Topic 3: Iterator

Why: To traverse collections safely.

What: An object used to iterate through elements of a collection.

How: Use Iterator or ListIterator with hasNext(), next(), remove().

When: When you need sequential access to elements.

Who: Developers working with dynamic collections.

Code Example (Iterator):

```
import java.util.*;
```

```
public class TestIterator {
```

```

public static void main(String[] args) {
    ArrayList<String> list = new ArrayList<>();
    list.add("A");
    list.add("B");
    list.add("C");

    Iterator<String> it = list.iterator();
    while(it.hasNext()) {
        System.out.println(it.next());
    }
}

```

Code Example (ListIterator):

```

import java.util.*;

public class TestListIterator {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.add("X");
        list.add("Y");
        list.add("Z");

        ListIterator<String> it = list.listIterator();

        while(it.hasNext()) {
            System.out.println(it.next());
        }

        while(it.hasPrevious()) {

```

```
        System.out.println(it.previous());
    }
}
}
```

Exercise:

Use an Iterator to remove all odd numbers from an ArrayList<Integer>.

Quiz:

Q3: Which method checks if more elements are available in an Iterator?

- a) next()
- b) hasNext()
- c) previous()
- d) remove()



Day 7 – Comparable, Comparator, Threads & Character Streams

Topic 1: Comparable & Comparator

Why: Sorting is a common requirement in applications.

What:

- **Comparable** → interface that allows natural ordering (compareTo).
- **Comparator** → interface that allows custom ordering (compare).
How: Implement Comparable in a class or create a separate Comparator.
When: Use Comparable for default sorting, Comparator for flexible sorting.
Who: Developers working with collections that need sorting.

Code Example (Comparable):

```
import java.util.*;

class Student implements Comparable<Student> {
    String name;
    int age;
```

```

Student(String name, int age) {
    this.name = name;
    this.age = age;
}

public int compareTo(Student s) {
    return this.age - s.age; // sort by age
}
}

public class TestComparable {
    public static void main(String[] args) {
        List<Student> list = new ArrayList<>();
        list.add(new Student("Alice", 22));
        list.add(new Student("Bob", 20));

        Collections.sort(list);
        for(Student s : list) {
            System.out.println(s.name + " " + s.age);
        }
    }
}

```

Code Example (Comparator):

```

import java.util.*;

class Student {
    String name;

```

```

int age;

Student(String name, int age) {
    this.name = name;
    this.age = age;
}
}

public class TestComparator {
    public static void main(String[] args) {
        List<Student> list = new ArrayList<>();
        list.add(new Student("Alice", 22));
        list.add(new Student("Bob", 20));

        Comparator<Student> nameComparator = (s1, s2) ->
s1.name.compareTo(s2.name);

        Collections.sort(list, nameComparator);
        for(Student s : list) {
            System.out.println(s.name + " " + s.age);
        }
    }
}

```

Exercise:

Create a list of employees and sort them by salary using Comparator.

Quiz:

Q1: Which interface defines natural ordering?

- a) Comparator
- b) Comparable
- c) Collection

- d) Sorter

Topic 2: Introduction to Threads

Why: To perform multiple tasks simultaneously.

What: A thread is a lightweight process.

How: Extend Thread class or implement Runnable.

When: When tasks can run concurrently (e.g., downloading files, animations).

Who: Developers building responsive applications.

Code Example (Extending Thread):

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread running...");  
    }  
}
```

```
public class TestThread {  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```

Code Example (Implementing Runnable):

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Runnable running...");  
    }  
}
```

```
public class TestRunnable {  
    public static void main(String[] args) {  
        Thread t = new Thread(new MyRunnable());  
        t.start();  
    }  
}
```

Exercise:

Create two threads: one prints even numbers, the other prints odd numbers.

Quiz:

Q2: Which method starts a thread?

- a) run()
 - b) start()
 - c) execute()
 - d) begin()
-

Topic 3: Thread Methods

Why: To control thread execution.

What: Common methods include sleep(), join(), start(), run().

How: Call these methods on thread objects.

When: To manage thread lifecycle.

Who: Developers working with concurrent applications.

Code Example:

```
class MyThread extends Thread {  
    public void run() {  
        for(int i=1; i<=5; i++) {  
            System.out.println(i);  
            try {  
                Thread.sleep(500); // pause for 0.5 sec  
            } catch(Exception e) {}  
        }  
    }  
}
```



```

    }
}

public class TestThreadMethods {

    public static void main(String[] args) {

        MyThread t1 = new MyThread();

        MyThread t2 = new MyThread();

        t1.start();

        try {

            t1.join(); // wait for t1 to finish

        } catch (Exception e) {}

        t2.start();

    }

}

```

Exercise:

Create a thread that prints numbers 1–10 with a delay of 1 second.

Quiz:

Q3: Which method pauses a thread for a given time?

- a) wait()
- b) sleep()
- c) join()
- d) stop()

Topic 4: Character Streams

Why: To read/write text files efficiently.

What: Classes like FileReader, FileWriter, BufferedReader, BufferedWriter.

How: Use these classes to handle character data.

When: When working with text files.

Who: Developers handling file I/O.

Code Example (FileReader & FileWriter):

```
import java.io.*;

public class TestCharStream {

    public static void main(String[] args) {

        try {

            FileWriter fw = new FileWriter("test.txt");

            fw.write("Hello Java");

            fw.close();

            FileReader fr = new FileReader("test.txt");

            int i;

            while((i = fr.read()) != -1) {

                System.out.print((char)i);

            }

            fr.close();

        } catch(IOException e) {

            e.printStackTrace();

        }

    }

}
```

Code Example (BufferedReader & BufferedWriter):

```
import java.io.*;

public class TestBuffered {

    public static void main(String[] args) {
```

```

try {

    BufferedWriter bw = new BufferedWriter(new FileWriter("buffer.txt"));

    bw.write("Buffered Writer Example");

    bw.close();


    BufferedReader br = new BufferedReader(new FileReader("buffer.txt"));

    String line;

    while((line = br.readLine()) != null) {

        System.out.println(line);

    }

    br.close();

} catch(IOException e) {

    e.printStackTrace();

}

}
}

```

Exercise:

Write a program to copy content from one text file to another using `BufferedReader` and `BufferedWriter`.

Quiz:

Q4: Which class reads text line by line?

- a) `FileReader`
- b) `BufferedReader`
- c) `FileWriter`
- d) `BufferedWriter`

Topic 1: File Handling

Why: To store and retrieve data permanently outside of program memory.

What: Java provides File class and streams (FileInputStream, FileOutputStream, FileReader, FileWriter).

How: Use these classes to create, read, write, and delete files.

When: When applications need persistent storage (e.g., logs, configs).

Who: Developers working with desktop apps, servers, or data processing.

Code Example (File class):

```
import java.io.File;

public class TestFile {

    public static void main(String[] args) {

        File f = new File("example.txt");

        if(f.exists()) {

            System.out.println("File exists: " + f.getName());

        } else {

            System.out.println("File does not exist");

        }

    }

}
```

Code Example (FileInputStream & FileOutputStream):

```
import java.io.*;

public class TestFileStream {

    public static void main(String[] args) {

        try {

            FileOutputStream fos = new FileOutputStream("data.txt");

            fos.write("Hello Java".getBytes());

            fos.close();

        }

    }

}
```

```

        FileInputStream fis = new FileInputStream("data.txt");

        int i;

        while((i = fis.read()) != -1) {

            System.out.print((char)i);

        }

        fis.close();

    } catch(IOException e) {

        e.printStackTrace();

    }

}
}

```

Exercise:

Write a program to copy contents from one file to another using `FileInputStream` and `FileOutputStream`.

Quiz:

Q1: Which class is used to represent file properties?

- a) `FileReader`
- b) `File`
- c) `FileWriter`
- d) `BufferedReader`

Topic 2: JDBC API Overview

Why: To connect Java applications with databases.

What: JDBC (Java Database Connectivity) provides APIs for database operations.

How: Use `DriverManager`, `Connection`, `Statement`, `PreparedStatement`.

When: When applications need to store/retrieve structured data.

Who: Developers building enterprise applications.

Code Example (JDBC Connection):

```
import java.sql.*;

public class TestJDBC {

    public static void main(String[] args) {

        try {

            Connection con = DriverManager.getConnection(

                "jdbc:mysql://localhost:3306/testdb", "root", "password");

            System.out.println("Connected to database");

            con.close();

        } catch (Exception e) {

            e.printStackTrace();

        }

    }

}
```

Exercise:

Write a program to connect to a database and print all table names.

Quiz:

Q2: Which class manages database drivers?

- a) Connection
- b) DriverManager
- c) Statement
- d) ResultSet

Topic 3: CRUD Operations

Why: CRUD (Create, Read, Update, Delete) are the basic database operations.

What: SQL queries executed via JDBC.

How: Use Statement or PreparedStatement.

When: Anytime data needs to be manipulated.

Who: Developers working with databases.

Code Example (Insert & Select):

```
import java.sql.*;

public class TestCRUD {

    public static void main(String[] args) {

        try {

            Connection con = DriverManager.getConnection(

                "jdbc:mysql://localhost:3306/testdb", "root", "password");

            // Insert

            PreparedStatement ps = con.prepareStatement("INSERT INTO students VALUES
            (?, ?)");

            ps.setInt(1, 1);

            ps.setString(2, "Alice");

            ps.executeUpdate();

            // Select

            Statement st = con.createStatement();

            ResultSet rs = st.executeQuery("SELECT * FROM students");

            while(rs.next()) {

                System.out.println(rs.getInt(1) + " " + rs.getString(2));

            }

            con.close();

        } catch (Exception e) {

            e.printStackTrace();

        }

    }

}
```

Exercise:

Write a program to update a student's name in the database using PreparedStatement.

Quiz:

Q3: Which interface is used to execute parameterized queries?

- a) Statement
 - b) PreparedStatement
 - c) ResultSet
 - d) Connection
-

Topic 4: Transactions

Why: To ensure data consistency during multiple operations.

What: Transactions group operations into a single unit.

How: Use commit(), rollback(), savepoint().

When: When multiple queries must succeed or fail together.

Who: Developers working with financial or critical systems.

Code Example:

```
import java.sql.*;

public class TestTransaction {

    public static void main(String[] args) {

        try {

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb", "root", "password");

            con.setAutoCommit(false);

            Statement st = con.createStatement();

            st.executeUpdate("INSERT INTO students VALUES (2, 'Bob')");
            st.executeUpdate("INSERT INTO students VALUES (3, 'Charlie')");

            con.commit(); // commit both inserts
```



```
        con.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}
```

Exercise:

Write a program that inserts two records but rolls back if the second insert fails.

Quiz:

Q4: Which method undoes all changes in a transaction?

- a) commit()
- b) rollback()
- c) savepoint()
- d) close()



Day 9 – Modern Java Features

Topic 1: Lambda Expressions

Why: To simplify code by writing functions in a concise way.

What: A lambda is an anonymous function (no name) used to implement functional interfaces.

How: Syntax → (parameters) -> expression.

When: When you need short, inline implementations (e.g., event handling, iteration).

Who: Developers working with functional programming in Java 8+.

Code Example:

```
import java.util.*;
```

```
public class TestLambda {
```

```
    public static void main(String[] args) {
```

```
        List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
```

```
        names.forEach(n -> System.out.println(n)); // Lambda expression
    }
}
```

Exercise:

Write a lambda expression to filter even numbers from a list of integers.

Quiz:

Q1: Lambda expressions work with which type of interface?

- a) Abstract class
 - b) Functional interface
 - c) Marker interface
 - d) Serializable
-

Topic 2: Stream API

Why: To process collections in a functional style.

What: Streams allow operations like filter, map, reduce.

How: Use stream() method on collections.

When: When working with large datasets or needing clean, declarative code.

Who: Developers handling data transformations.

Code Example:

```
import java.util.*;

public class TestStream {

    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(1,2,3,4,5,6);

        numbers.stream()

            .filter(n -> n % 2 == 0)

            .map(n -> n * n)

            .forEach(System.out::println); // prints squares of even numbers

    }
}
```

```
}
```

Exercise:

Use Stream API to count how many names in a list start with the letter "A".

Quiz:

Q2: Which Stream method is used to transform elements?

- a) filter
- b) map
- c) reduce
- d) collect

Topic 3: Optional Class

Why: To avoid NullPointerException.

What: A container object that may or may not hold a non-null value.

How: Use Optional.of(), Optional.empty(), isPresent(), orElse().

When: When dealing with values that may be null.

Who: Developers writing safe, null-free code.

Code Example:

```
import java.util.Optional;
```

```
public class TestOptional {  
    public static void main(String[] args) {  
        Optional<String> name = Optional.of("Alice");  
        System.out.println(name.orElse("Default"));  
  
        Optional<String> emptyName = Optional.empty();  
        System.out.println(emptyName.orElse("Default")); // prints Default  
    }  
}
```

Exercise:

Write a program that uses Optional to safely handle a null string input.

Quiz:

Q3: Which method provides a default value if Optional is empty?

- a) get()
 - b) orElse()
 - c) isPresent()
 - d) empty()
-

Topic 4: Generics

Why: To write reusable code that works with different data types.

What: Generics allow type parameters in classes, methods, and interfaces.

How: Use angle brackets <T>.

When: When building collections or utility methods.

Who: Developers writing flexible, type-safe code.

Code Example (Generic Class):

```
class Box<T> {  
  
    private T value;  
  
    public void set(T value) { this.value = value; }  
  
    public T get() { return value; }  
  
}
```

```
public class TestGenerics {  
  
    public static void main(String[] args) {  
  
        Box<String> b1 = new Box<>();  
  
        b1.set("Hello");  
  
        System.out.println(b1.get());  
  
  
        Box<Integer> b2 = new Box<>();  
  
        b2.set(100);  
  
        System.out.println(b2.get());  
  
    }  
}
```

```
}  
}
```

Code Example (Generic Method):

```
public class TestGenericMethod {  
    public static <T> void printArray(T[] array) {  
        for(T element : array) {  
            System.out.print(element + " ");  
        }  
    }  
}
```

```
    public static void main(String[] args) {  
        String[] names = {"Alice", "Bob"};  
        Integer[] numbers = {1,2,3};  
  
        printArray(names);  
        printArray(numbers);  
    }  
}
```

Exercise:

Create a generic class Pair<K,V> to store key-value pairs and print them.

Quiz:

Q4: Which symbol is used to define generics?

- a) {}
- b) []
- c) <>
- d) ()



Topic 1: Annotations

Why: To provide metadata about code that can be used by the compiler or runtime.

What: Annotations are special markers like @Override, @Deprecated, or custom ones.

How: Place @AnnotationName above classes, methods, or fields.

When: When you want to give instructions to the compiler or frameworks.

Who: Developers working with frameworks (Spring, Hibernate) or writing clean, maintainable code.

Code Example (Built-in Annotations):

```
class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound() { // ensures overriding is correct  
        System.out.println("Dog barks");  
    }  
}
```

```
@Deprecated  
void oldMethod() {  
    System.out.println("This method is deprecated");  
}  
}
```

Code Example (Custom Annotation):

```
import java.lang.annotation.*;
```

```

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface MyAnnotation {
    String value();
}

public class TestAnnotation {
    @MyAnnotation(value="Custom Annotation Example")
    public void display() {
        System.out.println("Display method");
    }

    public static void main(String[] args) throws Exception {
        TestAnnotation obj = new TestAnnotation();
        obj.display();

        // Access annotation using reflection
        MyAnnotation ann = obj.getClass()
            .getMethod("display")
            .getAnnotation(MyAnnotation.class);
        System.out.println("Annotation value: " + ann.value());
    }
}

```

Exercise:

Create a custom annotation @Info with fields author and version. Apply it to a class and print values using reflection.

Quiz:

Q1: Which annotation ensures a method overrides a parent method?

- a) @Deprecated

- b) @Override
- c) @SuppressWarnings
- d) @Retention

Topic 2: Design Patterns

Why: To provide proven solutions to common software design problems.

What: Reusable templates like Singleton, Factory, Observer, Strategy.

How: Implement patterns using classes, interfaces, and relationships.

When: When solving recurring design challenges.

Who: Developers building scalable, maintainable systems.

Singleton Pattern

What: Ensures only one instance of a class exists.

Code Example:

```
class Singleton {  
  
    private static Singleton instance;  
  
    private Singleton() {}  
  
  
  
    public static Singleton getInstance() {  
        if(instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}  
  
public class TestSingleton {  
  
    public static void main(String[] args) {  
        Singleton s1 = Singleton.getInstance();  
    }  
}
```



```
Singleton s2 = Singleton.getInstance();

System.out.println(s1 == s2); // true

}

}
```

Exercise:

Create a Singleton class DatabaseConnection that returns the same instance every time.

Factory Pattern

What: Creates objects without exposing instantiation logic.

Code Example:

```
interface Shape {

    void draw();

}
```

```
class Circle implements Shape {

    public void draw() { System.out.println("Circle drawn"); }

}
```

```
class Rectangle implements Shape {

    public void draw() { System.out.println("Rectangle drawn"); }

}
```

```
class ShapeFactory {

    public Shape getShape(String type) {

        if(type.equalsIgnoreCase("Circle")) return new Circle();

        else if(type.equalsIgnoreCase("Rectangle")) return new Rectangle();

        return null;

    }

}
```

```
}
```

```
public class TestFactory {  
    public static void main(String[] args) {  
        ShapeFactory factory = new ShapeFactory();  
        Shape s = factory.getShape("Circle");  
        s.draw();  
    }  
}
```

Exercise:

Create a factory that returns different types of Vehicle objects (Car, Bike).

Observer Pattern

What: Defines a one-to-many dependency between objects.

Code Example:

```
import java.util.*;
```

```
interface Observer {  
    void update(String message);  
}
```

```
class Subject {  
    private List<Observer> observers = new ArrayList<>();  
  
    void addObserver(Observer o) { observers.add(o); }  
    void notifyObservers(String msg) {  
        for(Observer o : observers) o.update(msg);  
    }  
}
```

```
class User implements Observer {  
    String name;  
    User(String name) { this.name = name; }  
    public void update(String message) {  
        System.out.println(name + " received: " + message);  
    }  
}
```

```
public class TestObserver {  
    public static void main(String[] args) {  
        Subject subject = new Subject();  
        User u1 = new User("Alice");  
        User u2 = new User("Bob");  
  
        subject.addObserver(u1);  
        subject.addObserver(u2);  
  
        subject.notifyObservers("New update available!");  
    }  
}
```

Exercise:

Implement an observer system where WeatherStation notifies multiple DisplayDevices.

Strategy Pattern

What: Defines a family of algorithms, encapsulates each, and makes them interchangeable.

Code Example:

```
interface PaymentStrategy {
```

```
    void pay(int amount);  
}
```

```
class CreditCardPayment implements PaymentStrategy {  
    public void pay(int amount) {  
        System.out.println("Paid " + amount + " using Credit Card");  
    }  
}
```

```
class PayPalPayment implements PaymentStrategy {  
    public void pay(int amount) {  
        System.out.println("Paid " + amount + " using PayPal");  
    }  
}
```

```
class ShoppingCart {  
    private PaymentStrategy strategy;  
    void setPaymentStrategy(PaymentStrategy strategy) {  
        this.strategy = strategy;  
    }  
    void checkout(int amount) {  
        strategy.pay(amount);  
    }  
}
```

```
public class TestStrategy {  
    public static void main(String[] args) {  
        ShoppingCart cart = new ShoppingCart();  
    }  
}
```

```
    cart.setPaymentStrategy(new CreditCardPayment());  
    cart.checkout(100);  
  
    cart.setPaymentStrategy(new PayPalPayment());  
    cart.checkout(200);  
}  
}
```

Exercise:

Create a strategy pattern for different sorting algorithms (BubbleSort, QuickSort).

Quiz:

Q2: Which pattern ensures only one instance of a class exists?

- a) Factory
- b) Singleton
- c) Observer
- d) Strategy

Q3: Which pattern is used to notify multiple objects of changes?

- a) Factory
 - b) Singleton
 - c) Observer
 - d) Strategy
-



Day 11 & 12 – Advanced Java 8 Features, Collections Framework & Generics

Topic 1: Java 8 Features – Streams & Optionals

Why: To simplify data processing and avoid null-related errors.

What:

- **Streams** → process collections in a functional style.

- **Optionals** → container to handle null values safely.
How: Use stream() for pipelines, Optional.of(), Optional.empty().
When: When working with large datasets or nullable values.
Who: Developers building modern, clean, and safe applications.

Code Example (Streams):

```
import java.util.*;

public class TestStreams {

    public static void main(String[] args) {

        List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "Anna");

        long count = names.stream()

            .filter(n -> n.startsWith("A"))

            .count();

        System.out.println("Names starting with A: " + count);

    }

}
```

Code Example (Optionals):

```
import java.util.Optional;

public class TestOptional {

    public static void main(String[] args) {

        Optional<String> name = Optional.ofNullable(null);

        System.out.println(name.orElse("Default Name"));

    }

}
```

Exercise:

Use Streams to find the maximum number in a list. Use Optional to handle the case when the list is empty.

Quiz:

Q1: Which method in Optional provides a default value?

- a) get()
- b) orElse()
- c) isPresent()
- d) empty()

Topic 2: Asynchronous & Parallel Programming in Java 8

Why: To improve performance by running tasks concurrently.

What:

- **CompletableFuture** → handles async tasks.
- **Parallel Streams** → process data in parallel.
How: Use CompletableFuture.supplyAsync() or parallelStream().
When: When tasks are independent and can run simultaneously.
Who: Developers optimizing performance.

Code Example (CompletableFuture):

```
import java.util.concurrent.*;

public class TestAsync {

    public static void main(String[] args) {

        CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {

            System.out.println("Running async task...");

        });

        future.join(); // wait for completion

    }

}
```

Code Example (Parallel Stream):

```
import java.util.*;

public class TestParallelStream {

    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(1,2,3,4,5,6,7,8);

        numbers.parallelStream()

            .forEach(n -> System.out.println(n + " " + Thread.currentThread().getName()));

    }

}
```

Exercise:

Use CompletableFuture to fetch two tasks in parallel and combine their results.

Quiz:

Q2: Which class is used for asynchronous programming in Java 8?

- a) Future
- b) CompletableFuture
- c) Thread
- d) Runnable

Topic 3: Collections Framework (Data Structures)

Why: To manage groups of objects efficiently.

What: Interfaces like List, Set, Map with implementations (ArrayList, HashSet, HashMap).

How: Use methods like add(), remove(), contains().

When: Anytime structured data needs to be stored and manipulated.

Who: Every Java developer.

Code Example (List & Set):

```
import java.util.*;
```



```
public class TestCollections {  
    public static void main(String[] args) {  
        List<String> list = new ArrayList<>();  
        list.add("A");  
        list.add("B");  
  
        Set<String> set = new HashSet<>(list);  
        set.add("C");  
  
        System.out.println("List: " + list);  
        System.out.println("Set: " + set);  
    }  
}
```

Exercise:

Create a Map<Integer, String> to store roll numbers and student names. Print all entries.

Quiz:

Q3: Which collection allows duplicate elements?

- a) Set
- b) List
- c) Map
- d) None

Topic 4: Generics (Advanced Use Cases)

Why: To write type-safe, reusable code.

What: Generics allow parameterized types in classes, methods, and interfaces.

How: Use <T>, <K,V>, wildcards (?).

When: When building reusable libraries or collections.

Who: Developers writing flexible, type-safe APIs.

Code Example (Generic Class with Wildcards):

```
class Printer<T> {
```

```

void print(T value) {
    System.out.println(value);
}
}

public class TestGenerics {
    public static void main(String[] args) {
        Printer<String> p1 = new Printer<>();
        p1.print("Hello");

        Printer<Integer> p2 = new Printer<>();
        p2.print(123);
    }
}

```

Code Example (Generic Method with Wildcards):

```

import java.util.*;

public class TestWildcard {
    public static void printList(List<?> list) {
        for(Object obj : list) {
            System.out.println(obj);
        }
    }

    public static void main(String[] args) {
        List<String> names = Arrays.asList("Alice", "Bob");
        printList(names);
    }
}

```

}

Exercise:

Create a generic class `Pair<K,V>` and store key-value pairs. Print them using a generic method.

Quiz:

Q4: Which symbol is used for wildcards in generics?

- a) *
- b) ?
- c) <>
- d) {}

Encapsulation vs Abstraction in Java

Aspect	Encapsulation	Abstraction
Why	To protect data and control access	To hide implementation details and expose only essential features
What	Wrapping variables and methods together inside a class, restricting direct access	Defining abstract classes or interfaces that specify <i>what</i> should be done, not <i>how</i>
How	Use access modifiers (private, public, protected) and getters/setters	Use abstract keyword for classes/methods or interface for contracts
When	When sensitive data must be hidden or controlled	When designing frameworks, APIs, or reusable components
Who	Developers ensuring secure, maintainable code	Developers defining blueprints or contracts for subclasses
Focus	Data hiding	Implementation hiding
Example	Private balance field in BankAccount with public deposit() and getBalance() methods	Abstract Shape class with abstract draw() method implemented by Circle and Rectangle

Aspect	Encapsulation	Abstraction
Keyword	private, public, protected	abstract, interface
Analogy	Locking valuables in a box and giving controlled access	Showing only the “remote control” buttons without revealing how the TV works inside
